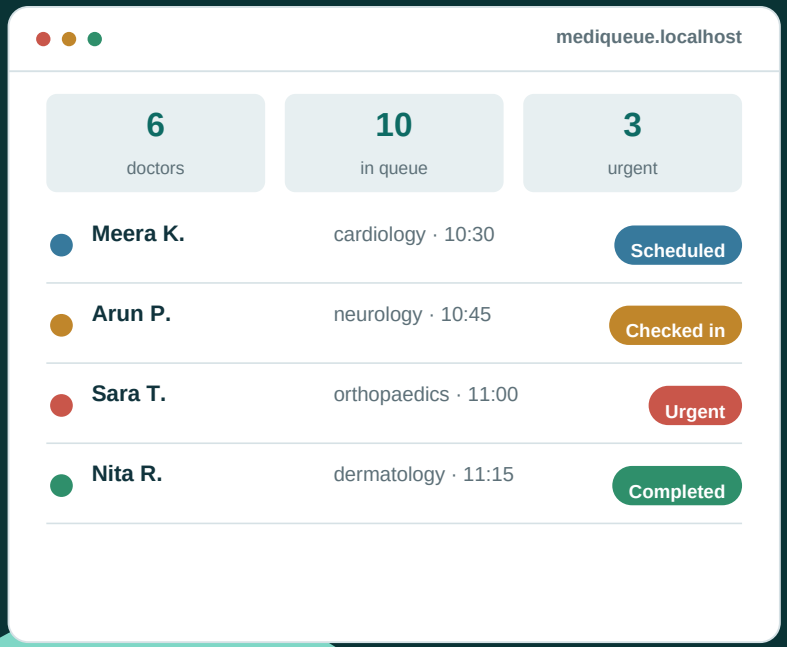
MediQueue

Clinic Appointment Desk

DevOps final capstone — application, CI/CD, infrastructure, Kubernetes, GitOps and observability

React 19 · FastAPI · PostgreSQL 17 · Docker · Kubernetes · Helm · Terraform · Prometheus · Grafana · Argo CD



Khushi Sarawagi · 24BCS10361

github.com/khushi-infinity/mediqueue

What This Covers

17 topics

1 Project Overview

2 The Problem It Solves

3 Architecture

4 Tech Stack

5 Application Data Model

6 Testing And Code Quality

7 Containers

8 CI/CD Pipeline

9 Security: Trivy Gate

10 Kubernetes And Helm

11 Observability

12 GitOps With Argo CD

13 Terraform: AWS Infrastructure

14 Evidence: It Runs

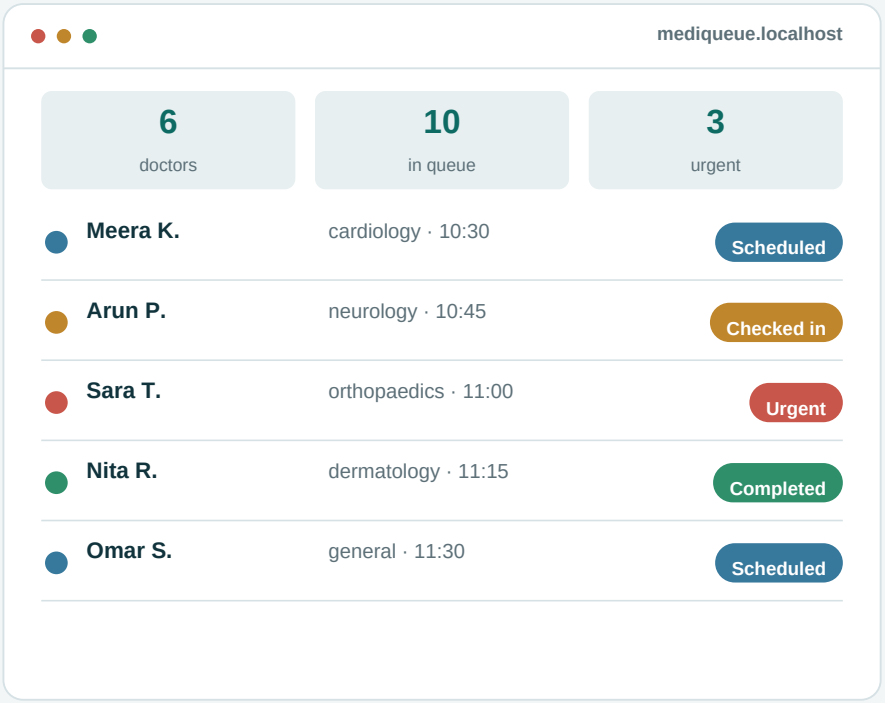
15 Troubleshooting Lab

16 Known Limits

17 Key Learnings

Project Overview

what the front desk actually does



6
doctors on the desk

11
API routes

7
status + urgency levels

The API behind it

Every screen is one call: /api/doctors, /api/appointments, /api/stats.

Filters, search and pagination are server-side, so the UI stays thin.

OpenAPI docs are generated, and the browser tests read them.

Who it is for

Reception books, checks in, completes or cancels visits.

Doctors are a first-class resource; every appointment references one.

Six departments share one queue.

What it shows

A KPI strip, status tabs, search and stable pagination.

Urgency is explicit, so the sickest patient is visible first.

Every view is an API call, so the same data feeds the dashboard.

Why this domain

An original clinic queue, built for the capstone.

Not a TaskBoard reskin: two related resources, not one flat list.

11 routes with full CRUD plus /api/stats.

The Problem It Solves

before · after

Paper diary and phone calls

Nobody can see the whole queue at once.

Two staff members can book the same slot.

How many patients are still waiting? No shared answer.

Urgency lives in the receptionist's head.

Nothing is queryable, so nothing can be measured.

— one queue →

One shared queue in MediQueue

Every visit is a row with a status the whole desk can see.

Departments belong to doctors, so they are never duplicated.

Urgency is explicit: routine, soon, urgent.

KPI strip answers 'how many are waiting' at a glance.

The same API feeds the UI, the tests and the metrics.

Duplicate bookings

gone — a visit is one row with one status

The waiting room

visible — the KPI strip answers it at a glance

Urgency

explicit — routine, soon or urgent on every row

MediQueue keeps one queue that the front desk, the API and the dashboard all read from the same place.

Architecture

commit SHA is the artifact identity

Developer
git push

GitHub · main
the desired state

GitHub Actions
test · build · scan · verify

GHCR
images tagged with the commit SHA

values-release.json
committed by CI, [skip ci]

Argo CD
sync + self-heal, pruning off

helm apply

Kubernetes — Kind locally · EKS on AWS

Ingress
host mediqueue.localhost

routes /api

frontend
2 replicas · nginx, UID 101

backend
2 replicas · FastAPI, UID 10001

scrape

DATABASE_URL

postgres
StatefulSet · PersistentVolumeClaim · UID 70

Prometheus
scrape every 10s · /metrics

queries

Grafana
provisioned dashboard · 2 alert rules

One identity end to end

The image tag is the commit SHA that built it.

Argo CD applies the exact SHA CI already verified on Kind.

Nothing is deployed that has not passed every gate behind it.

The same image is built, scanned, verified on an ephemeral Kind cluster, published, promoted and then reconciled — one SHA end to end.

Tech Stack

eight layers, one repository

Frontend	React 19 · Vite · Lucide icons · hand-written responsive CSS
API	Python 3.13 · FastAPI · Pydantic · SQLAlchemy 2
Data	PostgreSQL 17 · Alembic migrations
Quality	pytest (26 cases) · Ruff · Playwright (17 checks)
Containers	non-root Alpine images · multi-stage Nginx frontend
CI/CD	GitHub Actions · Trivy gate · GHCR by commit SHA
Infrastructure	Terraform (VPC + EKS) · Helm · Kubernetes
Observability	Prometheus · Grafana · Argo CD GitOps

Application Data Model

two related resources, one real foreign key

doctors — 5 columns

id	PK
full_name	
department	
specialization	
active	

1 — one doctor, many visits — N

appointments — 9 columns

id	PK
doctor_id	FK → doctors.id
patient_name	
patient_phone	
scheduled_for	
reason	
status	
urgency	
notes	

Design decisions

Department belongs to the doctor, so it is never repeated per visit.

doctor_id is a real foreign key, so a visit cannot outlive its doctor.

Status and urgency are validated enums, not free text.

Constraints & indexes

Primary key on both tables; appointments.doctor_id is indexed.

List filters escape wildcards, so a search for % is a search for %.

Pagination is stable because the sort ends on a unique column.

status

scheduled · checked_in · completed · cancelled

urgency

routine · soon · urgent

Migrations

Alembic owns the schema; CI runs alembic upgrade head against real PostgreSQL before pytest.

API surface

11 routes: /api/doctors and /api/appointments with full CRUD, plus /api/stats and /healthz.

Freshness

updated_at is set by the API, so the desk can see how long a patient has been waiting.

Testing And Code Quality

no image is built until these pass

Ruff
import order · bugbear rules · 110-column limit

pytest
26 cases against PostgreSQL in CI, SQLite locally

frontend build
the production bundle must compile

helm lint
the chart must be valid before it ships

terraform validate
no cloud resources are created

26
pytest API cases

17
Playwright checks

What the tests cover

CRUD on both resources and every route group

filters, stats and pagination stability

validation, 404 and 409 conflict paths

wildcard escaping in list filters

readiness failing when the database is gone

metric cardinality: route templates, never raw IDs

Local and in CI

Locally pytest uses SQLite, so a laptop needs no database.

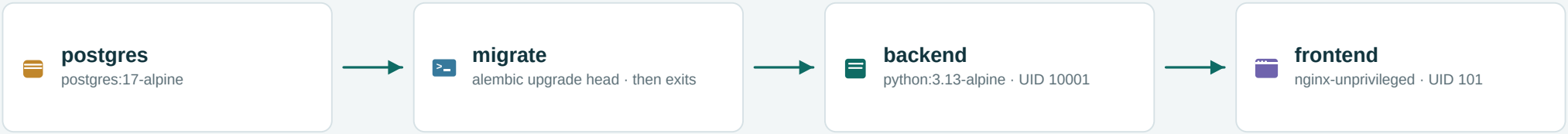
In CI the same tests run against PostgreSQL 17 first.

Playwright drives the real frontend through the Ingress.

A failing gate stops the run; nothing is published.

Containers

the same images run in Compose and in the cluster



Backend image

Multi-stage build; pip is removed from the runtime layer.

Runs as UID 10001, not root.

Uvicorn workers behind a readiness probe.

Frontend image

node:24 builds the bundle; Nginx serves it.

nginx-unprivileged listens as UID 101, not root.

/healthz is a real endpoint, so the probe is honest.

Runs as non-root

No container in the stack needs root to start.

No secrets baked in

Credentials arrive as environment variables at deploy time, never in a layer.

Readiness, not hope

Traffic only reaches pods that report ready, so a rollout cannot serve 503s.

Compose healthchecks gate each service on the previous one being ready; the migrate service must finish before the API starts.

CI/CD Pipeline

run 38755163712 — green in five minutes

Job 1 · test

ruff → alembic + pytest → frontend build → helm lint → terraform validate

Job 2 · build-scan-deploy

build images (SHA) → Trivy ×2 → Kind deploy + smoke → Playwright → push to GHCR → promote SHA

the gate

Guarantees

No continue-on-error on any quality or security gate.

Images are tagged with the full commit SHA, never latest.

The Kind cluster is ephemeral: created and thrown away per run.

Superseded runs cancel themselves, so main is never queued behind a stale build.

Promotion commits carry [skip ci], so a release cannot loop.

What the run produced

26 tests and 17 browser checks, zero failures.

Both Trivy scans clean: alpine and every Python package.

Both images published to GHCR with a recorded digest.

helm template asserts the exact image references before the deploy step waits eight minutes.

The deployed chart is verified on Kind before anything is published.

5	2	0	8
minutes, end to end	jobs, test then deploy	retries needed	minute deploy budget

Security: The Trivy Gate

Static analysis (SAST)	reads source
Dependency scanning	known CVEs in libraries
Secret scanning	credentials in the tree
Container image scanning	Trivy — the layer this project runs
Runtime security	what runs at deploy time

The gate

severity: HIGH,CRITICAL

exit-code: 1 — a finding fails the build

vuln-type: os,library on both images

Runs after the build, before any deploy.

What it does not prove

Business logic and authorisation are not scanned.

A clean image is not a clean application.

No runtime or network policy is enforced here.

0

HIGH or CRITICAL

2/2

images scanned

No secrets in the tree

Database and Grafana credentials are generated at deploy time and never committed.

Non-root containers

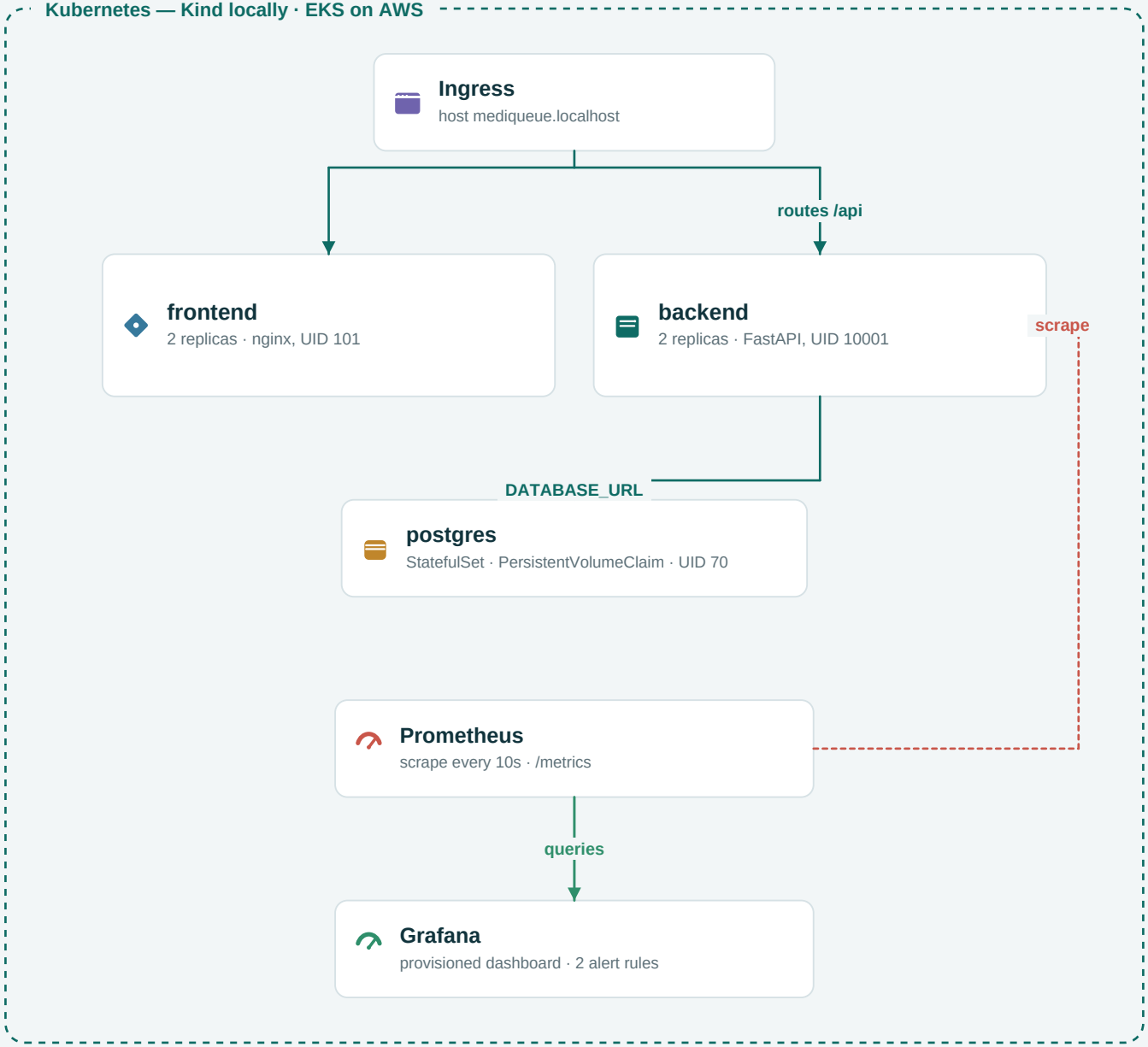
Both images run as an unprivileged UID, so a container escape starts with nothing.

Prove, then publish

An image is scanned and deployed to Kind before it is ever pushed to the registry.

No credentials are committed; every local secret is generated at deploy time.

Kubernetes And Helm



Helm rendering

1 chart + values files produce 16 objects.

Values are merged in order, later files win.

values.yaml → values-release.json (CI) → values-local.yaml.

Scheduling

Readiness gates endpoint membership, so traffic only reaches ready pods.

HPA targets 70% CPU across 2–4 replicas; 2 → 3 was observed under load.

PostgreSQL is a StatefulSet with a PVC; the rest is stateless.

16 rendered objects

Deployments and Services for both tiers.

A StatefulSet with its PersistentVolumeClaim for PostgreSQL.

Ingress, ConfigMap, ServiceAccount and the HorizontalPodAutoscaler.

A one-shot Job runs the migration before the API rolls out.

16
objects, one chart

2–4
replicas, HPA

Observability

scrape every replica, not just one

backend ×2
/metrics

headless Service
discovers replicas

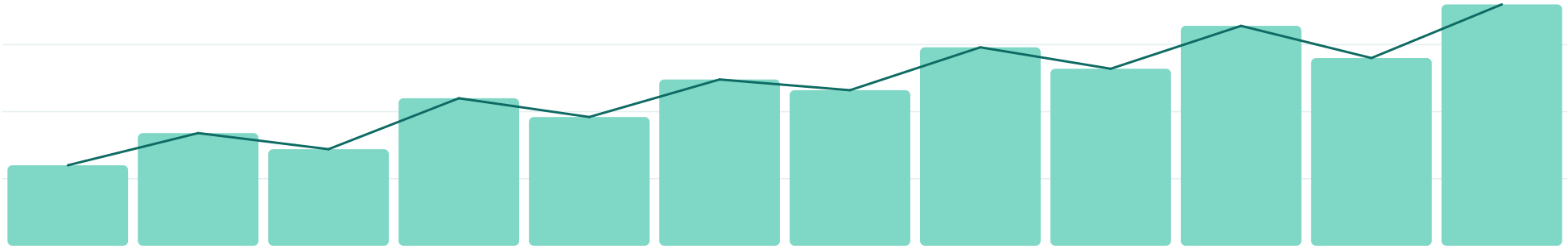
Prometheus
scrape every 10s

Grafana + alerts
dashboard and rules

MediQueue / Service Health

provisioned in Grafana

2 / 2	12.4 /s	41 ms	0.0 %
healthy replicas	request rate	p95 latency	5xx ratio



What we measure

Request rate and latency per route template.

5xx ratio, so a broken deploy is visible in seconds.

Live replica count against the HPA target.

Database readiness, because a queue with no store is down.

Two alert rules

One fires when no backend replica is scrapeable.

One fires on a sustained 5xx ratio.

Both were observed INACTIVE on the live stack.

Locally there is no notification target, so they only change state.

Counters are labelled with route templates, never raw IDs, so metric cardinality stays bounded.

GitOps With Argo CD



Default branch

main is the only source of truth; Argo CD tracks the branch, never a hand-made tag.

Sync policy

automated with self-heal, so drift is corrected without anyone noticing it happened.

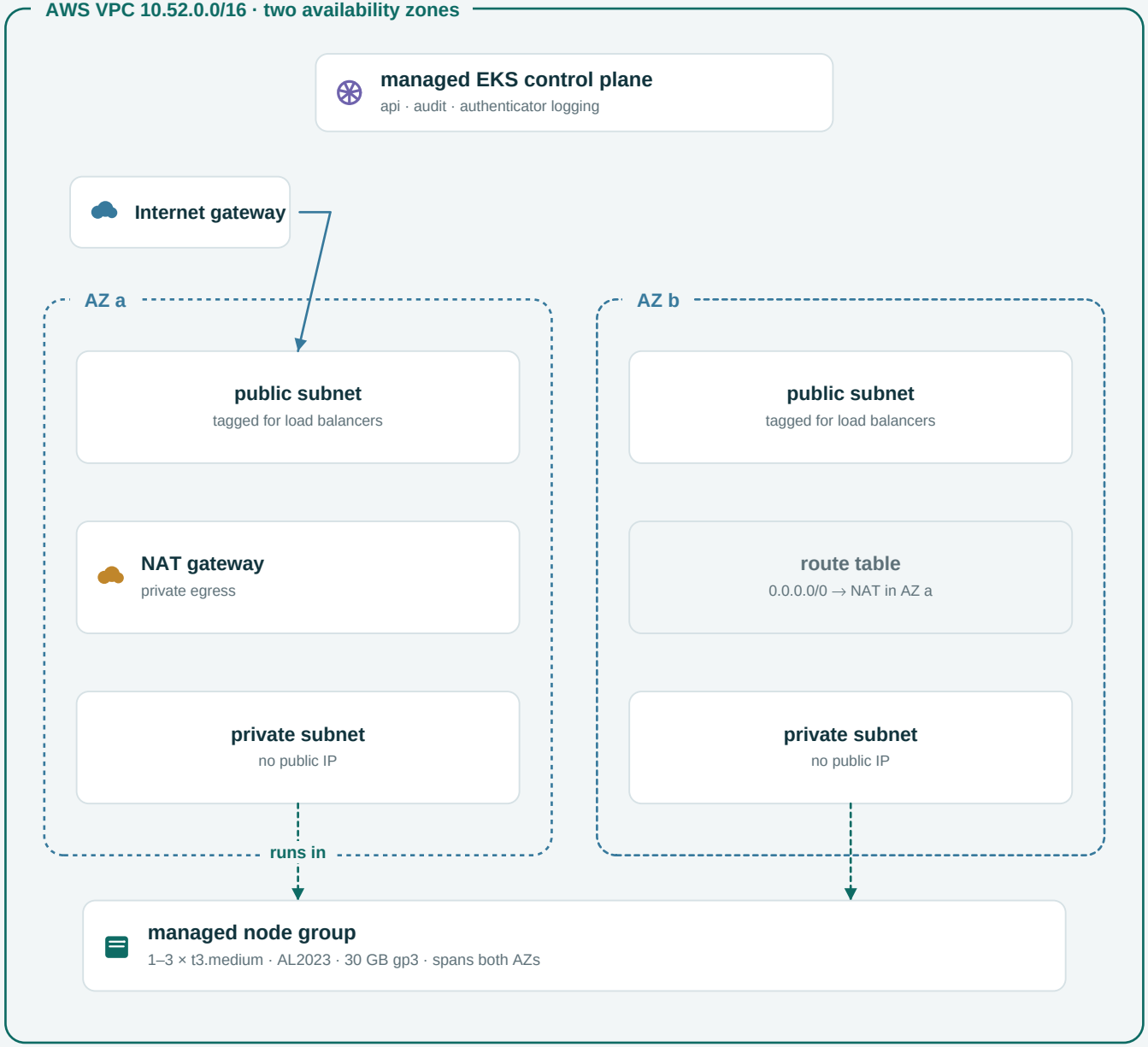
Pruning off

a bad commit cannot delete the database or its volume, so the worst case is a stalled sync.

Promotion commits carry [skip ci] so a release never re-triggers an image build.

Terraform: AWS Infrastructure

written and validated, apply deferred



Infrastructure as code is reviewable; console clicks are not.

What the plan creates

VPC 10.52.0.0/16 across two availability zones

two public and two private subnets, tagged for load balancers

one NAT gateway for private egress

managed EKS control plane with api/audit/authenticator logging

managed node group: 1–3 × t3.medium, AL2023, 30 GB

Guardrails

EBS CSI driver via IRSA, so PVCs can actually bind.

The EKS API is restricted to supplied CIDRs.

0.0.0.0/0 is rejected by input validation.

How it is verified

terraform fmt -check and terraform validate run in CI.

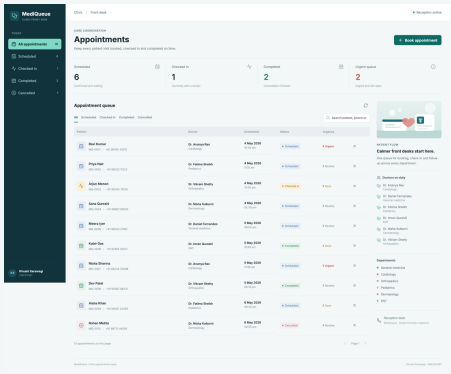
The plan is reviewed as code, not as console clicks.

No apply is run, so no AWS account has been touched.

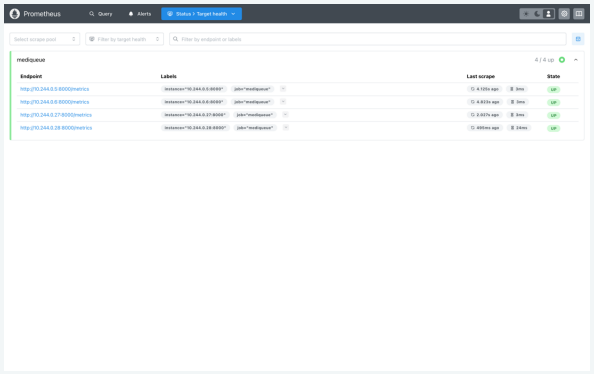
Applying and destroying is a single documented command.

Evidence: It Runs

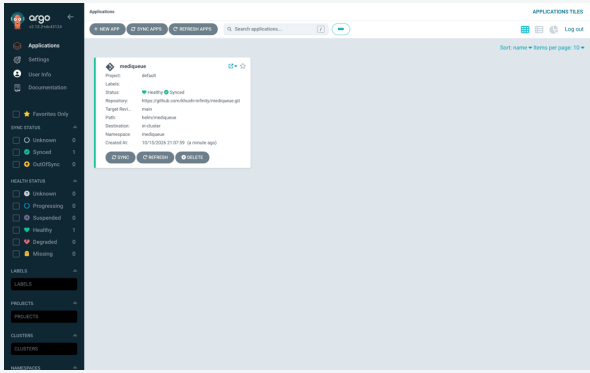
captured on this machine and in CI



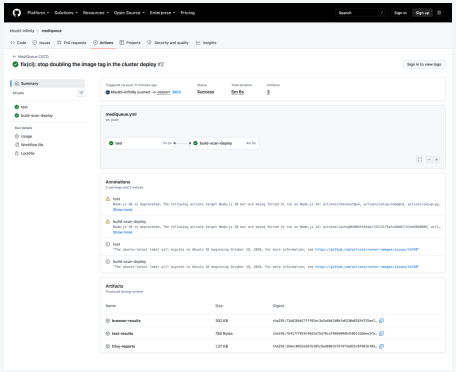
- **Served through the Ingress hostname**



- **Both backend replicas UP**



- **Argo CD synced from GitHub**



- **Pipeline green: both jobs passed**

26	17	0	2
tests passed	browser checks	CVEs found	images published

Troubleshooting Lab

scripts/troubleshooting-check.py

SYMPTOM	FIRST COMMAND	ROOT CAUSE	FIX
ImagePullBackOff	`kubectl describe pod`	a tag that does not exist	correct the tag, watch the rollout
Service has no endpoints	`kubectl get endpoints`	selector matches no pod labels	fix the selector or the labels
/ready returns 503	`kubectl logs <pod>`	the migration never ran	run the Job, then let readiness pass
HPA will not scale	`kubectl top pods`	no CPU request on the container	set resource requests, then re-check

The lesson that matters

helm rollback restores chart configuration, not database migrations — schema changes need their own, forward-only path.



Known Limits

stated plainly, not discovered later

No authentication or RBAC

High

Anyone who can reach the ingress can read and change every appointment. Keep it local, or add auth before it holds real data.

Alerts have no destination

Low

The rules fire and change state, but nothing is notified until a receiver is configured.

Single PostgreSQL instance

Medium

No backups, no failover and no encryption policy. A lost volume loses the whole queue.

AWS apply deferred

Info

Terraform is written and validated only; no account has been touched, so the EKS path is unproven in practice.

Ephemeral monitoring storage

Low

Prometheus and Grafana keep their data in the pod, so history is lost on every restart.

Key Learnings

what the work actually taught

1. A deployment is only as trustworthy as the gates in front of it.
2. Readiness is a different question from liveness, and it changes routing.
3. Immutable SHA tags make production traceable back to a commit.
4. Metrics must be bounded: route templates, not IDs, keep dashboards sane.
5. Infrastructure as code is reviewable; console clicks are not.
6. One owner per deployment avoids Helm and Argo CD fighting each other.